
Test-Time Compute Allocation: Search vs. Verify

Tianxiang Xie

Dalian University of Technology
xietianxiang@mail.dlut.edu.cn

Jingxuan Wu

Dalian University of Technology
wujingxuan@mail.dlut.edu.cn

Jiaying Liu

Dalian University of Technology
jiaying_liu@outlook.com

Shuo Yu

Dalian University of Technology
shuo.yu@ieee.org

The Concept: The concept introduced in this paper is *test-time compute allocation*: the problem of dividing a fixed inference budget (a fixed number of model calls) between search and verification. Search aggregates many candidate answers by majority voting; verification generates fewer and checks each with a judge. Search carries a mathematical guarantee: with independent candidates and accuracy p above $1/2$, the failure probability of majority voting falls exponentially in N ; verification has no such guarantee, and only helps when the judge can tell good answers from bad ones. In this course, the judge is the model grading its own answers; our measurements show it is weak. Under the weak-judge, moderate-budget setting of this course, search usually wins; the best split depends on your model, your judge, and your budget. Our measured setting is in Table 1.

Table 1: Measured accuracy on the 100-problem MATH 500 subset (Qwen3-4B, three seeds, budget 50 calls per problem).

Strategy	Split	Accuracy [95% CI]	Reference
Search	50 candidates, majority vote	0.853 [0.81, 0.89]	[4] (Majority)
Search + light verification	25 candidates $\times$ 1 check	0.717 [0.67, 0.77]	–
Verification	5 candidates $\times$ 9 checks	0.753 [0.70, 0.80]	[4] (PRM weighted)

Leveling and Prerequisite Knowledge: To follow this material, learners need three foundations: how an LLM generates answers (autoregressive decoding, one forward pass per token), enough probability for the binomial reasoning behind majority voting, and what the MATH and GSM8K benchmarks measure. The material supplies these foundations itself: three self-check questions in Notebook 00 verify them.

Level 1: no prerequisites. Then learners need only the core trade-off in plain terms from the five-minute video.

Level 2 (systematic) is aimed at graduate students and early-career researchers in LLM reasoning and is adaptable for advanced undergraduates; it adds the math and the hands-on experiments.

Learning Objectives: By the end of this material, learners will be able to: **Level 0 (Foundation):** explain what test-time compute allocation is (dividing a fixed inference budget between search and verification) and why the split matters. **Level 1 (Advanced):** explain the search–verification trade-off in plain language, naming the two conditions under which search wins (a weak judge, a moderate budget). **Level 2 (Ultimate goal):** (1) estimate a model’s failure rate with a simplified majority-voting model and predict the candidate count N needed; (2) plot a budget–accuracy curve and read the six-model curve family; (3) given a new task and fixed budget, reason out the best search–verify split from the model’s accuracy and the judge’s strength; (4) judge whether a research paper’s baselines are fair.

Linked Papers: Test-time compute allocation is in active use at the frontier; each paper below builds on it. *Scaling LLM Test-Time Compute Optimally* [4] shows compute-optimal splits beat fixed best-of- N by up to $4\times$, and with a strong judge (e.g., a trained reward model) verification-guided

search wins at low to moderate budgets. *Sample, Scrutinize and Scale* [5] pushes the allocation toward verification at very large budgets. *Let Me Think!* [2] shows one long chain can be worth exponentially many short ones. *Provable Scaling Laws* [1] gives pairwise-comparison aggregation a guarantee even below $p = 1/2$. *When to Solve, When to Verify* [3] finds self-consistency (search) more compute-efficient than verification for most budgets, exactly the weak-judge regime this course measures.

Teaching Materials Summary: All materials are original and were created for the NeurIPS 2026 Education Track; everything lives in <https://github.com/Solitus0726/search-vs-verify-course>, containing: (1) video; (2) slides; (3) notebooks 00–04 (self-check, three experiments, report template); (4) scripts and pre-computed data; (5) the strategy diagram and the five reference papers.

Suggested order: start with the self-check (Notebook 00), then follow the materials in the order shown in Figure 1. Shorter routes for limited time: an intuition-first pass (about 30 minutes) follows the video, the strategy diagram, and Notebook 01 in simplified mode; a quick reference (about 10 minutes) starts from the strategy diagram and jumps to the relevant notebook section. All routes end at the acceptance checks. Notebooks 01–03 plot the budget–accuracy curves, find the optimal split under the budget identity $N \times (1 + M) = 50$, and predict the best split for GSM8K; Notebook 04 is the report template. The slides and strategy diagram add the formal math and a one-page visual summary. The scripts and data make every result reproducible: one to two hours from scratch, or instantly with the pre-computed cache.

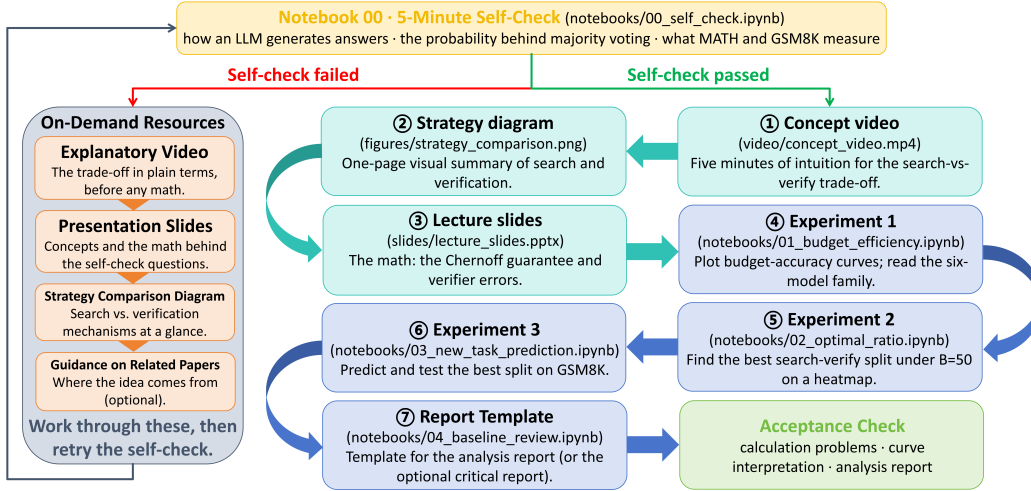


Figure 1: Learning path: a five-minute self-check routes to the on-demand resources if it fails, or to the main sequence if it passes.

References

- [1] Yanxi Chen, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. Provable Scaling Laws for the Test-Time Compute of Large Language Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=GBMzJLhsRj>.
- [2] Parsa Mirtaheri, Ezra Edelman, Samy Jelassi, Eran Malach, and Enric Boix-Adserà. Let Me Think! A Long Chain of Thought Can Be Worth Exponentially Many Short Ones. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 166795–166826. Curran Associates, Inc., 2025. doi: 10.52202/085713-5561. URL <https://openreview.net/forum?id=GuvQJGgbLm>.
- [3] Nishad Singhi, Hritik Bansal, Arian Hosseini, Aditya Grover, Kai-Wei Chang, Marcus Rohrbach, and Anna Rohrbach. When To Solve, When To Verify: Compute-Optimal Problem Solving and Generative Verification for LLM Reasoning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=R7qRUFHGTx>.
- [4] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning. In *International Conference on Learning Representations*, volume 2025, pages 10131–10165, 2025. URL <https://openreview.net/forum?id=4FWAwZtd2n>.
- [5] Eric Zhao, Pranjal Awasthi, and Sreenivas Gollapudi. Sample, Scrutinize and Scale: Effective Inference-Time Search by Scaling Verification. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 77272–77309. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/zhao25a.html>.